Offline Speaker Diarization — Detailed Guide

A complete, self-contained explanation: what the project does, how the pipeline works end-to-end, every key parameter, and how to run & tune it. No prior knowledge assumed.

1 · WHAT THIS PROJECT DOES

Speaker diarization answers the question **"who spoke when?"**. Given a recording, it splits the audio into time segments and labels each with a speaker — `SPEAKER_00`, `SPEAKER_01`, … — so the *same voice always gets the same label*. It does **not** identify people by name (no "this is Alice"); it only tells apart *distinct voices* and tracks them consistently through the recording.

Input → Output

- **Input:** any audio/video file (or a media URL).
- **Output:** speaker count + a timeline of `(start, end, SPEAKER_XX)` blocks, as printed text, a JSON file, and a colored HTML timeline.

Two classic failures it avoids

- **Fragmentation:** one real person split into many phantom IDs.
- **Label permutation:** speaker labels randomly swapping over time.

Both are solved by using a **neural diarizer with a global view** of the whole file (details below).

2 · THE PIPELINE END-TO-END

The recording is decoded to **16 kHz mono PCM**, then passed through a neural model that internally slides a **10-second analysis window with a 1-second step** (so consecutive windows overlap by 90%). Within each window it finds speech and local speaker turns; a voice "fingerprint" is computed per turn; finally all fingerprints from the whole file are clustered into a small number of speakers. A short post-processing stage cleans the result into a readable timeline.

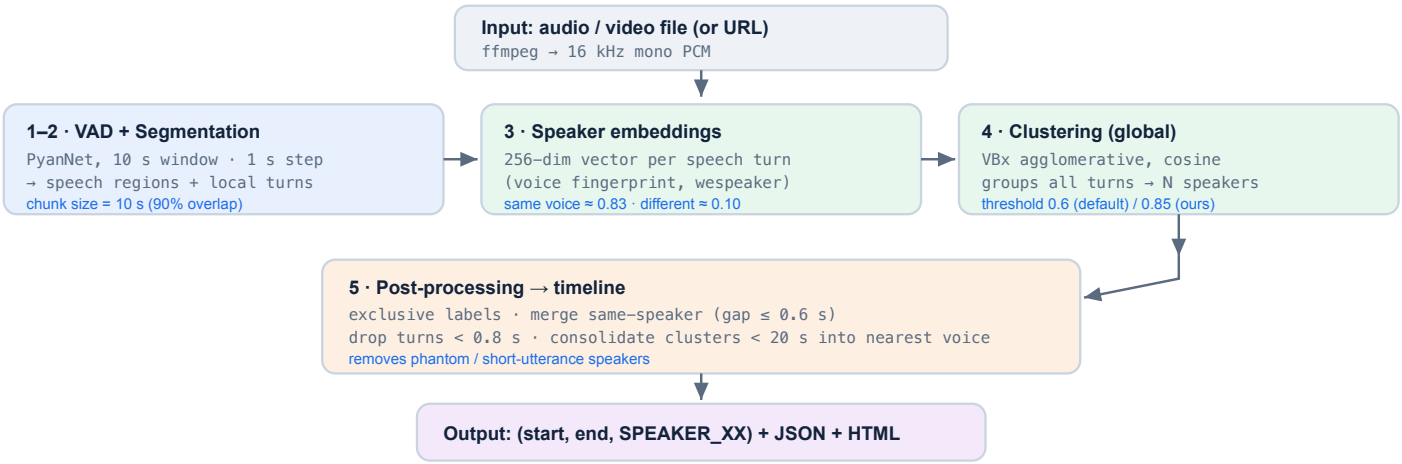

Fig. 1 — Full data flow. The neural model (stages 1–3) analyses 10 s chunks; clustering (4) merges them with a whole-file view.

3 · EACH STAGE EXPLAINED

Stage 1 — Voice Activity Detection (VAD)

Finds where *anyone* is speaking and discards silence, music and noise, so later stages only see speech. Handled by the same neural segmentation model.

Stage 2 — Segmentation (the "chunking")

The audio is scanned with a **10-second sliding window** that advances **1 second** at a time (the `step`), giving heavy overlap so no turn is missed at a boundary. Inside each window a neural network (**PyanNet**) predicts, frame by frame, which of up to a few local speakers is active — including **speaker-change points**. This is why boundaries are accurate even when people talk quickly back-and-forth.

Stage 3 — Speaker Embeddings (voice fingerprints)

For every speech turn, a neural speaker-verification network outputs a fixed **256-dimensional vector** that encodes the voice's characteristics. The crucial property: vectors from the **same person are close** (cosine similarity ≈ **0.83**) while vectors from **different people are far apart** (≈ **0.10**). This wide, reliable gap is what makes speakers separable.

Stage 4 — Clustering (the heart)

All embeddings from the entire file are grouped by a **VBx agglomerative clustering** algorithm using cosine similarity. Because it sees **every** turn at once (a "global view"), it can reliably decide how many speakers exist and which turns belong together — the step that defeats fragmentation and label-permutation. Its **threshold** controls how eagerly clusters merge (see §5).

Stage 5 — Post-processing → clean timeline

- **Exclusive labelling:** assign exactly one speaker to each instant.
- **Merge** consecutive same-speaker turns separated by ≤ **0.6 s**.
- **Drop** turns shorter than **0.8 s** from the timeline (noise/blips).
- **Consolidate** any cluster with less than **20 s** of total speech into its nearest real speaker by voice similarity — this removes "phantom" speakers created by short interjections ("yeah", "right").

4 · KEY CONCEPT — KNOWN VS UNKNOWN SPEAKER COUNT

Known count most reliable

Tell the model exactly how many speakers there are (`--speakers 2`). It removes all guesswork and gives the best accuracy — ideal for fixed formats (2-host podcast, panel with a set guest count).

Unknown count

The model estimates the number itself. Convenient, but count estimation is inherently error-prone; the two safeguards (threshold + consolidation) keep it clean. There is **no single threshold perfect for every recording** — an honest, well-known limitation.

5 · ALL PARAMETERS & DEFAULTS

Parameter	Default	Where	What it controls
Sample rate	16 kHz mono	input	required audio format for the models
`segmentation window`	10.0 s	pyannote (internal)	the analysis "chunk" size
`segmentation step`	1.0 s	pyannote (internal)	hop between chunks → 90% overlap
`embedding dim`	256	pyannote (internal)	size of each voice fingerprint
`clustering threshold`	0.6 std / **0.85** ours	**tuning knob**	higher = merge more (fights over-split); lower = split more
`Fa`, `Fb`	0.07, 0.8	VBx (advanced)	clustering smoothing — rarely changed
`num_speakers`	*None* (auto)	**tuning knob**	set an exact known count for best accuracy
`min_speaker_dur`	20 s (recommended)	**tuning knob**	merge clusters with less speech than this into nearest voice
`min_change_dur`	0.8 s	post-processing	drop turns shorter than this from the timeline
`merge gap`	0.6 s	post-processing	join adjacent same-speaker turns within this gap

6 · HOW TO RUN

One-command tool — prints a summary, writes `verify_result.json`, opens a colored `timeline.html`:

```
python verify.py interview.mp3 --speakers 2          # known count (best)
python verify.py podcast.wav                         # auto-detect count
python verify.py "https://youtu.be/<ID>" --speakers 2  # from a URL (needs yt-dlp)
```

As a library:

```
from diarizer import load_pipeline, diarize_file
pipe = load_pipeline()                       # auto: CUDA > Apple MPS > CPU
timeline, n = diarize_file("meeting.wav", num_speakers=None, min_speaker_dur=20.0)
# timeline = [(start_sec, end_sec, "SPEAKER_XX"), ...]
```

Reading the output

```
======================================================
  RESULT: 2 speaker(s)  |  7 speaker changes
======================================================
  #  1  00:00-00:31  (31.4s)  SPEAKER_01
  #  2  00:31-00:53  (21.7s)  SPEAKER_00   ...
```

7 · TUNING CHEAT-SHEET

Symptom	Fix
You know the exact number of speakers	`--speakers N` (always best)
One lively speaker split into extra IDs	raise `--threshold` (0.85 → 0.90)
Two similar voices merged into one	lower `--threshold` (0.85 → 0.70)
Tiny phantom speakers from short “yeah/right”	raise `--min-speaker-dur` (20 → 30)

8 · REQUIREMENTS & REPRODUCTION

- **Python 3.10+**, and **ffmpeg** on the system PATH.
- **GPU strongly recommended:** NVIDIA **CUDA** (or Apple **MPS**) is ~10–30× faster than CPU. Install the matching PyTorch build; the code auto-selects CUDA → MPS → CPU.
- **Model access (one-time):** a free Hugging Face token, and accept the terms for `pyannote/speaker-diarization-community-1`. Weights (~1–2 GB) download once and cache locally, then everything runs offline.
- Full step-by-step setup (incl. Windows) is in the project **README.md**.

9 · TECHNOLOGY & HONEST LIMITATIONS

Stack: `pyannote-audio 4` · `speaker-diarization-community-1` · wespeaker 256-d embeddings · `VBx` clustering · `PyTorch` · `ffmpeg` — all open-source, no paid APIs, no cloud.

- Unknown-count diarization has no universally perfect threshold — prefer a known count when possible.
- Two people who sound almost identical (e.g. same accent + pitch) may occasionally merge.
- A speaker who talks for only a few seconds total may be absorbed into another by consolidation.
- Heavy overlapping speech (people talking at once) is attributed to one speaker per instant.